

DSA -- Sorting Algorithms & Asymptotic Complexity

[Target] Target: State Big O / Omega / Theta in one line each. Give complexity of all 5 sorts cold.
Read time: 15 minutes

Asymptotic Notations

Asymptotic notations describe the behaviour of an algorithm and determine the rate of growth of a function as input size grows.

The 3 Notations

Notation
Symbol
What it bounds
Gives
Use

Big O
 $O(g(n))$
Upper bound
Worst case
Most commonly used

Omega
 $\omega(g(n))$
Lower bound
Best case
Theoretical min

Theta
 $\Theta(g(n))$
Tight bound
Average case
Both upper and lower

1. Big O Notation -- Upper Bound (Worst Case)

" $O(g(n))$ is the worst case -- no matter what input you give, the algorithm won't take longer than this."

Definition: $f(n) = O(g(n))$ if there exist positive constants C and n such that:

$0 \leq f(n) \leq C \cdot g(n)$ for all $n \geq n$

$f(n)$ is at most $C \cdot g(n)$ for large n .

2. Omega Notation -- Lower Bound (Best Case)

" $\omega(g(n))$ is the best case -- the algorithm takes at least this long."

Definition: $f(n) = \omega(g(n))$ if:

$C \cdot g(n) \leq f(n)$ $C \geq 0, n \geq 1$

3. Theta Notation -- Tight Bound (Average Case)

" $\Theta(g(n))$ means the algorithm is tightly bounded -- it's exactly this order of complexity."

Definition: $f(n) = \Theta(g(n))$ if:

$C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n)$ $C_1, C_2 \geq 0, n \geq n, n \geq 1$

Growth Rate Hierarchy (Slowest -- Fastest growing)

$O(1) \leq O(\log n) \leq O(n) \leq O(n \log n) \leq O(n^2) \leq O(2^n) \leq O(n!)$

Constant Log Linear Linearithmic Quadratic Exponential Factorial

Mnemonic (Hinglish): "Chota Log Nahi Manta Quadratic Exponential" Constant, Log, N, N-logN, Quadratic, Exponential

Case Analysis

Case
Meaning

Best Case
Input that causes minimum operations

Worst Case
Input that causes maximum operations

Average Case
Take all inputs, compute average operations

Sorting Algorithms -- Two Categories

Sorting

Comparison Based

Bubble Sort, Selection Sort, Insertion Sort

Quick Sort, Merge Sort, Heap Sort

Counting/Non-Comparison Based

Bucket Sort, Radix Sort

BUBBLE SORT

Idea: Compare adjacent elements and swap if out of order. Largest element "bubbles" to the end each pass.

Algorithm:

1. Start with unsorted array
2. For each pass: compare $arr[i]$ and $arr[i+1]$, swap if $arr[i] > arr[i+1]$
3. After each pass, largest unsorted element is in correct position
4. Repeat until no swaps needed (sorted flag)

Pass 1: [64, 34, 25, 12, 22, 11, 90] 90 bubbles to end

Pass 2: [34, 25, 12, 22, 11, 64, 90] 64 in position

...

Properties:

- In-place sorting (no extra array)
- Stable sorting [OK] (equal elements maintain relative order)
- Recursive relation: $T(n) = T(n-1) + n$

Case
Complexity

Best
 $O(n)$ -- already sorted, one pass to verify

Average
 $O(n^2)$

Worst
 $O(n^2)$ -- reverse sorted

Space
 $O(1)$

SELECTION SORT

Idea: Find the minimum element in the unsorted part, swap it to the front.

Algorithm:

1. Initialize $min_idx = 0$
2. Traverse unsorted part to find minimum

3. Swap minimum with element at min_idx

4. Increment min_idx and repeat

Properties:

- In-place sorting
- Unstable sorting -- min swap can skip over equal elements
- Max swaps = $n - 1$, Min swaps = 0
- Recursive relation: $T(n) = T(n-1) + (n-1)$

Case

Complexity

Best

$O(n)$ -- always scans full unsorted part

Average

$O(n)$

Worst

$O(n)$

Space

$O(1)$

Key point: Selection sort always does exactly $(n-1)$ comparisons per pass regardless of input -- no early exit.

INSERTION SORT

Idea: Build sorted array one element at a time by inserting each element into its correct position.

Algorithm:

1. Start from arr[1] (first element is trivially sorted)
2. Pick current element (key)
3. Compare key with sorted part to its left
4. Shift elements right until correct position found
5. Insert key at correct position

Properties:

- In-place, Stable [OK]
- Best for nearly-sorted data
- Insertion Sort = Position + Shifting
- Very sensitive to input order

Case

Complexity

Best

$O(n)$ -- already sorted, each element just compared once

Average

$O(n)$

Worst

$O(n)$ -- reverse sorted

Space

$O(1)$

Max comparisons = $n(n-1)/2$, Min comparisons = $n-1$
Recursive relation: $T(n) = T(n-1) + n$

QUICK SORT

Idea: Divide and conquer. Pick a pivot, partition array so elements smaller than pivot are left, larger are right. Recurse on both halves.

Properties:

- Divide and Conquer technique
- Unstable sorting
- In-place ($O(\log n)$ stack space)
- Recursive: $T(n) = 2T(n/2) + n$

Case
Complexity
When

Best
 $O(n \log n)$
Pivot always divides perfectly

Average
 $O(n \log n)$

Worst
 $O(n^2)$
Pivot always min or max (already sorted input)

Space
 $O(\log n)$
Recursion stack

Number of comparisons = $n(n-1)/2$
Master theorem gives: $(n \log n)$

MERGE SORT

Idea: Divide array into halves, sort each half recursively, then merge sorted halves.

Algorithm:

```
MergeSort(arr, l, r):  
  if r > l:  
    m = l + (r-l)/2  
    MergeSort(arr, l, m)    // sort left half  
    MergeSort(arr, m+1, r)  // sort right half  
    merge(arr, l, m, r)     // merge sorted halves
```

Properties:

- Pure Divide and Conquer
- Stable sorting [OK]
- Recursive: $T(n) = 2T(n/2) + n$

Case
Complexity
When

Best
 $O(n \log n)$

Always

Average
 $O(n \log n)$
Always

Worst
 $O(n \log n)$
Always (no bad case)

Space
 $O(n)$
Extra array for merging

Key advantage: Merge sort has guaranteed $O(n \log n)$ -- no worst-case $O(n^2)$ like Quick Sort.

RADIX SORT

Idea: Sort numbers digit by digit, from least significant to most significant digit.

Properties:

- Counting-based (non-comparison) sorting
- Outplace sorting (needs extra space)
- Stable [OK]

Condition
Complexity

$n \gg K$
 $O(n \cdot k)$

$n = K$
 $O(n^2)$

Space
 $O(n \cdot R)$ where R = radix (base)

Sorting -- Master Comparison Table

Algorithm
Best
Average
Worst
Space
Stable
Type

Bubble Sort
 $O(n^2)$
 $O(n^2)$
 $O(n^2)$
 $O(1)$
[OK]
Comparison

Selection Sort

$O(n^2)$

$O(n^2)$

$O(n^2)$

$O(1)$

Comparison

Insertion Sort

$O(n^2)$

$O(n^2)$

$O(n^2)$

$O(1)$

[OK]

Comparison

Quick Sort

$O(n \log n)$

$O(n \log n)$

$O(n)$

$O(\log n)$

Comparison

Merge Sort

$O(n \log n)$

$O(n \log n)$

$O(n \log n)$

$O(n)$

[OK]

Comparison

Heap Sort

$O(n \log n)$

$O(n \log n)$

$O(n \log n)$

$O(1)$

Comparison

Radix Sort

$O(nk)$

$O(nk)$

$O(nk)$

$O(n+k)$

[OK]

Counting

Mnemonic -- $O(n^2)$ sorts: "BSI = Bahut Slow hai India mein" (Bubble, Selection, Insertion)

Mnemonic -- $O(n \log n)$ sorts: "MQH = Meri Quick Ho gayi!" (Merge, Quick, Heap)

Recursion

A recurrence relation is an equation that describes a function in terms of its values on smaller inputs.

Solving a recurrence = finding the closed-form (non-recursive) time complexity.

4 Methods for Solving Recurrences

Method

When to use

Substitution

Guess the answer, then prove by induction

Recursion Tree

Draw the tree, sum costs at each level

Iteration

Expand the recurrence manually

Master Theorem

For divide-and-conquer recurrences $T(n) = aT(n/b) + f(n)$

Quick Reference -- Common Recurrences

Recurrence

Solution

$$T(n) = T(n-1) + 1$$

$$O(n)$$

$$T(n) = T(n-1) + 1$$

$$O(n)$$

$$T(n) = T(n-1) + \log n$$

$$O(n \log n)$$

$$T(n) = T(n-1) + n$$

$$O(n^2)$$

$$T(n) = T(n-2) + 1$$

$$O(n/2) = O(n)$$

$$T(n) = T(n-100) + n$$

$$O(n)$$

Master Theorem (Dividing Recurrences)

For: $T(n) = aT(n/b) + f(n)$ where $a \geq 1$, $b > 1$

Compare $f(n)$ with $n^{\log_b a}$:

Case

Condition

Solution

Case 1

$$f(n) \ll n^{\log_b a}$$

$$T(n) = \Theta(n^{\log_b a})$$

Case 2

$$f(n) = \Theta(n^{\log_b a})$$

$$T(n) = (n^{\log_b a} - \log n)$$

Case 3

$$f(n) > n^{\log_b a}$$

$$T(n) = (f(n))$$

Example -- Merge Sort: $T(n) = 2T(n/2) + n$

- $a=2, b=2, f(n)=n$

- $\log_2 2 = 1$, so $n^1 = n$

- $f(n) = n = n^{\log_b a}$ Case 2

- $T(n) = (n \log n)$ [OK]

Algorithm Criteria (5 Properties)

Every algorithm must satisfy:

Input Output Finiteness Definiteness Effectiveness

Property

Meaning

Input

Zero or more inputs

Output

At least one output

Finiteness

Must terminate after finite steps

Definiteness

Each step must be clearly defined

Effectiveness

Each step must be feasible

Mnemonic: "I O F D E = I'll Only Finish Definite Efforts"

Your 40-Second Script -- Sorting

"The main $O(n)$ sorts are Bubble, Selection, and Insertion. Bubble compares adjacent elements and swaps -- stable but slow. Selection finds the minimum each pass -- unstable. Insertion places each element in its correct position -- stable and fast on nearly-sorted data. The $O(n \log n)$ sorts are Merge, Quick, and Heap. Merge Sort is always $O(n \log n)$ but needs $O(n)$ space. Quick Sort is $O(n)$ worst case when pivot is always min or max. Heap Sort is always $O(n \log n)$ with $O(1)$ space."

Your 20-Second Script -- Asymptotic Notations

"Big O is the upper bound -- worst case. Omega is the lower bound -- best case. Theta is the tight bound -- average case. For example, Binary Search is $O(\log n)$ worst case, $\Omega(1)$ best case when found at mid, and $\Theta(\log n)$ on average."

Follow-Up Questions (Expect These)

Q: Why is Merge Sort preferred over Quick Sort in some scenarios?

Merge Sort has guaranteed $O(n \log n)$ -- no $O(n)$ worst case. Also stable. Preferred for linked lists and external sorting. Quick Sort is faster in practice for in-memory arrays due to cache performance.

Q: What makes Radix Sort faster than comparison sorts?

Radix Sort doesn't compare elements -- it sorts by digit positions. For fixed-length integers with small range, it achieves $O(n-k)$ which can beat $O(n \log n)$.

Q: Quick Sort is $O(n^2)$ worst case -- when does this happen?

When the pivot is always the minimum or maximum element -- happens on already sorted or reverse-sorted input with a naive pivot selection. Randomized pivot or median-of-three avoids this.

Q: What is Heapify and how is it different from inserting elements one by one?

Heapify builds the heap from an existing array in $O(n)$ time by applying sift-down from the last internal node upward. Inserting n elements one by one takes $O(n \log n)$. Heapify is faster.